



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Aula 7

Técnicas de Análise

Disciplina: ANÁLISE E PROJETO DE SISTEMAS

Prof. Emanuel Coutinho

Agenda

- Estereótipos
- Técnicas de análise
- Padrões

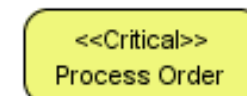
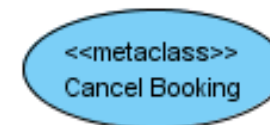
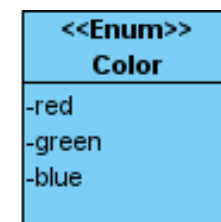
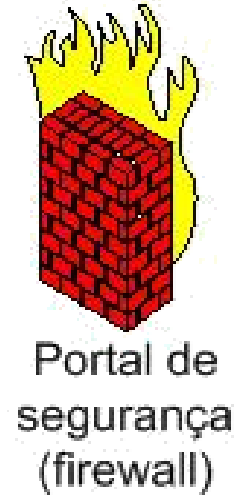
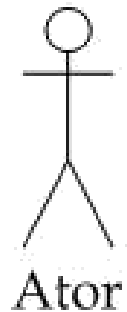


Estereótipos

Estereótipos

- Utilizado para estender (enriquecer) o significado de um determinado elemento em um diagrama
- UML predefine diversos estereótipos
- É possível também definir estereótipos específicos
- Estereótipos podem ser classificados em dois tipos:
 - estereótipo gráfico: um ícone que lembre o significado do conceito a ele associado.
 - estereótipos de rótulo: um nome delimitado pelos símbolos << e >>

Exemplos de estereótipos



Análise Textual de Abbott

Análise Textual de Abbott

- Estratégia:
 - Identificar termos da narrativa de casos de uso e documento de requisitos que podem sugerir classes, atributos, operações
- Utiliza diversas fontes de informação sobre o sistema:
 - Documento e requisitos, modelos do negócio, glossários, conhecimento sobre o domínio, etc

Análise Textual de Abbott

1. Para cada um desses documentos, nomes (substantivos e adjetivos) que aparecem no mesmo são destacados

Análise Textual de Abbott

2. Após isso, os sinônimos são removidos

- Permanecem os nomes mais significativos para o domínio do negócio em questão

Análise Textual de Abbott

3. Cada termo remanescente se encaixa em uma das situações a seguir:

- O termo se torna uma classe (classes candidatas)
- O termo se torna um atributo
- O termo não tem relevância para a modelagem

Análise Textual de Abbott

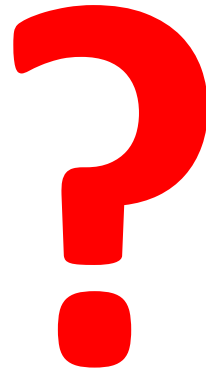
4. Identificação de operações e de associações

- Destacar verbos no texto
- Verbos de ação (ex: calcular, confirmar, cancelar, comprar, fechar, estimar, depositar, sacar, etc.) são operações em potencial
- Verbos com sentido de “ter” (tem-um) são potenciais agregações ou composições
- Verbos com sentido de “ser” (é-um) são generalizações em potencial
- Demais verbos são associações em potencial

Análise Textual de Abbott

- Aplicação bastante simples
- Desvantagem
 - Dependem:
 - Documentos utilizados como fonte serem completos, atualizado, corretos
 - Estilo de escrita
 - Muitos sinônimos
 - Variações linguísticas

Dúvidas





Exercícios

- Aplique a Análise Textual de Abbott para o Sistema de Reserva de Hotel

O Sistema de Reserva de Hotel permite que clientes realizem reservas de quartos. Cada reserva possui datas de check-in e check-out. O sistema calcula o valor total baseado na tarifa do quarto e noites de hospedagem. Funcionários do hotel podem gerenciar a disponibilidade dos quartos. O sistema envia confirmações por email para clientes.



Solução

- Cliente - nome, emailCliente, telefone, documento
- Reserva - idReserva, dataCheckIn, dataCheckOut, valorTotal, numeroNoites, status
- Quarto - numero, tipo, tarifaDiaria, disponível, descricao
- Funcionário - nome, cargo, idFuncionario
- Hotel - nomeHotel, localizacao
- Confirmação - idConfirmacao, dataEnvi, codigoReserva, conteudoEmail



Relacionamentos Identificados

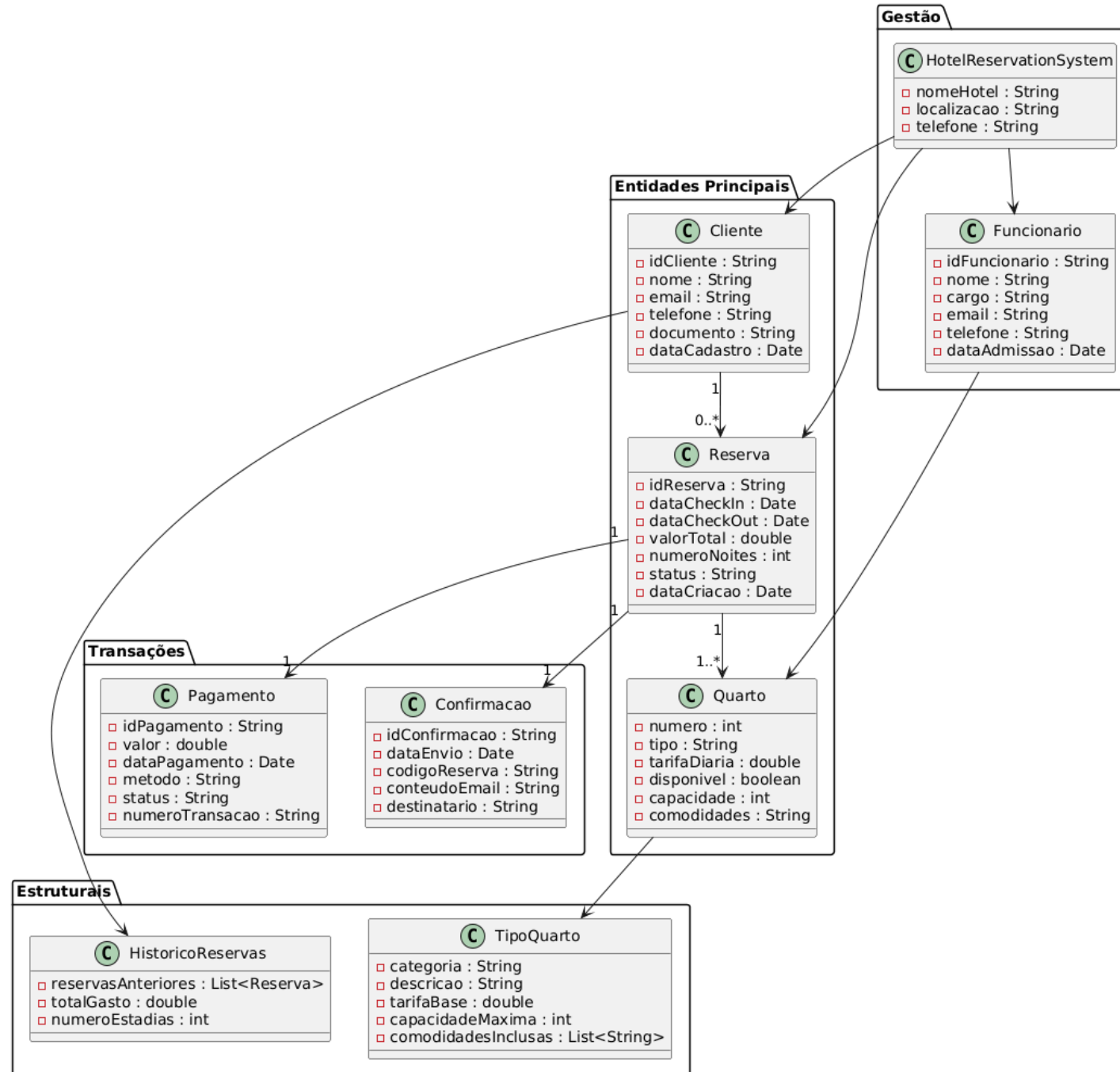
Relacionamento	Tipo	Descrição
Cliente - Reserva	Associação 1:N	Um cliente pode ter várias reservas
Reserva - Quarto	Associação N:1	Uma reserva está associada a um quarto
Funcionario - Quarto	Associação 1:N	Um funcionário gerencia vários quartos
Sistema - Reserva	Associação 1:N	O sistema gerencia várias reservas
Sistema - Cliente	Associação 1:N	O sistema tem vários clientes



Operações Mapeadas

Operação	Classe	Parâmetros	Retorno
realizarReserva()	Cliente	Quarto, Date, Date	void
calcularValor()	Reserva	-	double
gerenciarDisponibilidade()	Funcionario	Quarto, boolean	void
enviarConfirmacao()	HotelReservationSystem	Cliente, Reserva	void
verificarDisponibilidade()	Quarto	Date, Date	boolean
processarPagamento()	HotelReservationSystem	Reserva	boolean

Diagrama de Classes - Sistema de Reserva de Hotel (Organizado por Pacotes)



Análise de Casos de Uso

Análise de Casos de Uso

- Também chamada de identificação dirigida por Casos de Uso
- Ponto de partida é o Caso de Uso
- Premissa:
 - Um caso de uso corresponde a um comportamento específico do sistema
 - Esse comportamento somente pode ser produzido por objetos que compõem o sistema
 - A realização de um caso de uso é:
 - Responsabilidade de um conjunto de objetos
 - Objetos que devem colaborar para produzir o resultado daquele caso de uso

Análise de Casos de Uso

- Procedimento:
 - Verificação da descrição textual de cada caso de uso para identificar classes candidatas
 - Analisar cada caso de uso texto
 - Para cada caso de uso, identificar classes que possam fornecer o comportamento do mesmo
 - Quando todos os casos de uso tiverem sido analisados, as classes (ou pelo menos a grande maioria delas) terão sido identificadas

1. Pegue o fluxo de eventos do caso de uso como entrada;
2. Aplique a ATA em cada fluxo;
3. Remova Candidatos redundantes (sinônimos e etc);
4. Remova candidatos vagos;
5. Remova atores;
6. Separe os atributos;
7. Separe as operações.

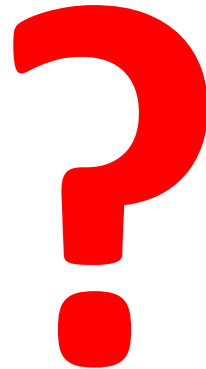
Identificando Atributos

- Descrição da classe
- Ex:
 - Carro
 - Marca, modelo, cor, valor de mercado

Identificando Operações

- Baseado nos verbos identificados
- Em geral, quatro categorias:
 - Manipulação de dados
 - Adicionar, remover, formatar, selecionar, etc
 - Exame do estado do objeto
 - Identificar a cor do carro
 - Execução de alguma computação
 - Calcular o valor do frete
 - Monitoramento da ocorrência de um evento de controle
 - Verificar se o funcionário se aposentou

Dúvidas





Exercícios

- Aplique a Análise de Casos de Uso para o Sistema de Reserva de Hotel

O Sistema de Reserva de Hotel permite que clientes realizem reservas de quartos. Cada reserva possui datas de check-in e check-out. O sistema calcula o valor total baseado na tarifa do quarto e noites de hospedagem. Funcionários do hotel podem gerenciar a disponibilidade dos quartos. O sistema envia confirmações por email para clientes.

Buscar Disponibilidade:

O Cliente seleciona buscar quartos

O Cliente informa as datas desejadas

O Sistema verifica a disponibilidade dos quartos e exibe uma lista de quartos disponíveis no período

Selecionar Quarto e Iniciar Reserva:

O Cliente seleciona um quarto

O Cliente preenche as informações para a reserva

O Sistema cria uma reserva prévia para o quarto selecionado

Calcular Valor Total:

O Sistema calcula o valor total da reserva ($\text{valor} = \text{tarifa_diária} * \text{número_de_noites}$)

Processar Pagamento:

O Sistema apresenta o valor da reserva ao Cliente

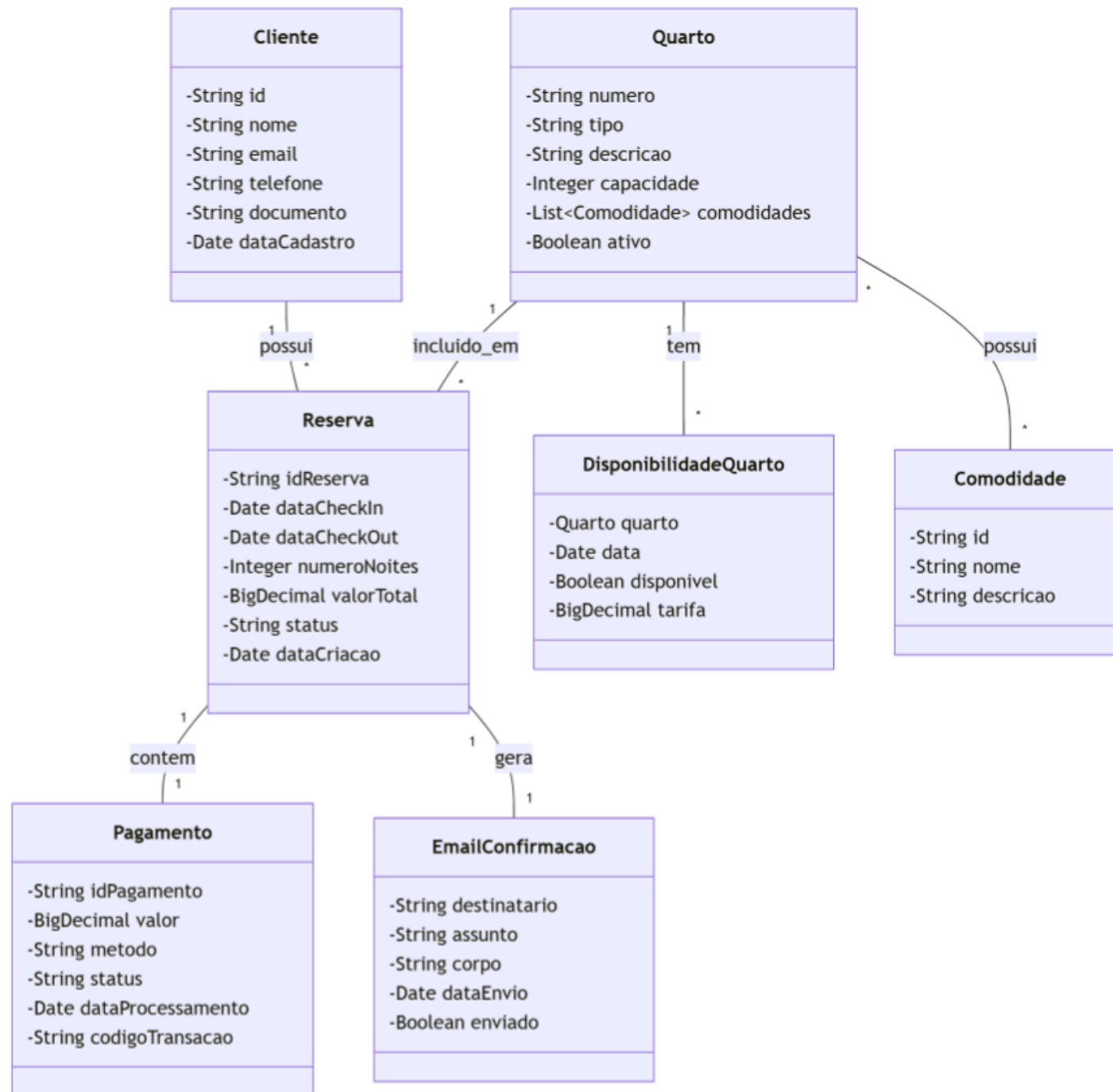
O Cliente informa os dados de pagamento

O Sistema processa o pagamento

O Sistema confirma a reserva

Enviar Confirmação:

O Sistema envia os detalhes da reserva para o cliente via e-mail



Métodos Principais Identificados

Classe Reserva

- `calcularNumeroNoites()` : Calcula o número de noites entre check-in e check-out
- `calcularValorTotal(BigDecimal tarifaDiaria)` : Calcula o valor total baseado na tarifa
- `confirmarReserva()` : Altera o status para CONFIRMADA

Classe DisponibilidadeQuarto

- `verificarDisponibilidade(Date dataInicio, Date dataFim)` : Verifica se o quarto está disponível no período
- `bloquearDisponibilidade(Date dataInicio, Date dataFim)` : Bloqueia o quarto para novas reservas

Classe SistemaReservas (Classe Controladora)

- `buscarQuartosDisponiveis(Date checkIn, Date checkOut)` : Retorna lista de quartos disponíveis
- `criarReservaPrevia(Quarto quarto, Cliente cliente, Date checkIn, Date checkOut)` : Cria reserva com status pendente
- `processarPagamento(Reserva reserva, DadosPagamento dados)` : Processa o pagamento da reserva
- `enviarEmailConfirmacao(Reserva reserva)` : Dispara e-mail de confirmação

Categorização BCE

Categorização BCE

- Objetos de um sistema são agrupados de acordo com o tipo de responsabilidade a eles atribuída
 - objetos de entidade: usualmente objetos do domínio do problema
 - objetos de fronteira: atores interagem com esses objetos
 - objetos de controle: servem como intermediários entre objetos de fronteira e de entidade, definindo o comportamento de um caso de uso específico

Categorização BCE

- Categorização proposta por Ivar Jacobson em 1992
 - Possui correspondência (mas não equivalência!) com o *model-view-controller* (MVC)
 - Ligação entre análise (o que; problema) e projeto (como; solução)
- Estereótipos na UML: «boundary», «entity», «control»



Objetos de Entidade

- Repositório para informações e as regras de negócio manipuladas pelo sistema
 - Representam conceitos do domínio do negócio
- Características
 - Normalmente armazenam informações persistentes
 - Várias instâncias da mesma entidade existindo no sistema
 - Participam de vários casos de uso e têm ciclo de vida longo
- Exemplo:
 - Um objeto **Pedido** participa dos casos de uso Realizar Pedido e Atualizar Estoque
 - Este objeto pode existir por diversos anos ou mesmo tanto quanto o próprio sistema



Objetos de Fronteira

- Realizam a comunicação do sistema com os atores
 - traduzem os eventos gerados por um ator em eventos relevantes ao/do sistema
 - responsáveis por apresentar os resultados de uma interação dos objetos em algo inteligível pelo ator
- Existem para que o sistema se comunique com o mundo exterior
 - Por consequência, são altamente dependentes do ambiente
- Há dois tipos principais de objetos de fronteira:
 - Os que se comunicam com o usuário (atores humanos): relatórios, páginas HTML, interfaces gráfica desktop, etc
 - Os que se comunicam com atores não-humanos (outros sistemas ou dispositivos): protocolos de comunicação



Objetos de Controle

- São a “ponte de comunicação” entre objetos de fronteira e objetos de entidade
- Responsáveis por controlar a lógica de execução correspondente a um caso de uso
- Decidem o que o sistema deve fazer quando um evento de sistema ocorre
 - Realizam o controle do processamento
 - Agem como gerentes (coordenadores, controladores) dos outros objetos para a realização de um caso de uso
- Traduzem eventos de sistema em operações que devem ser realizadas pelos demais objetos

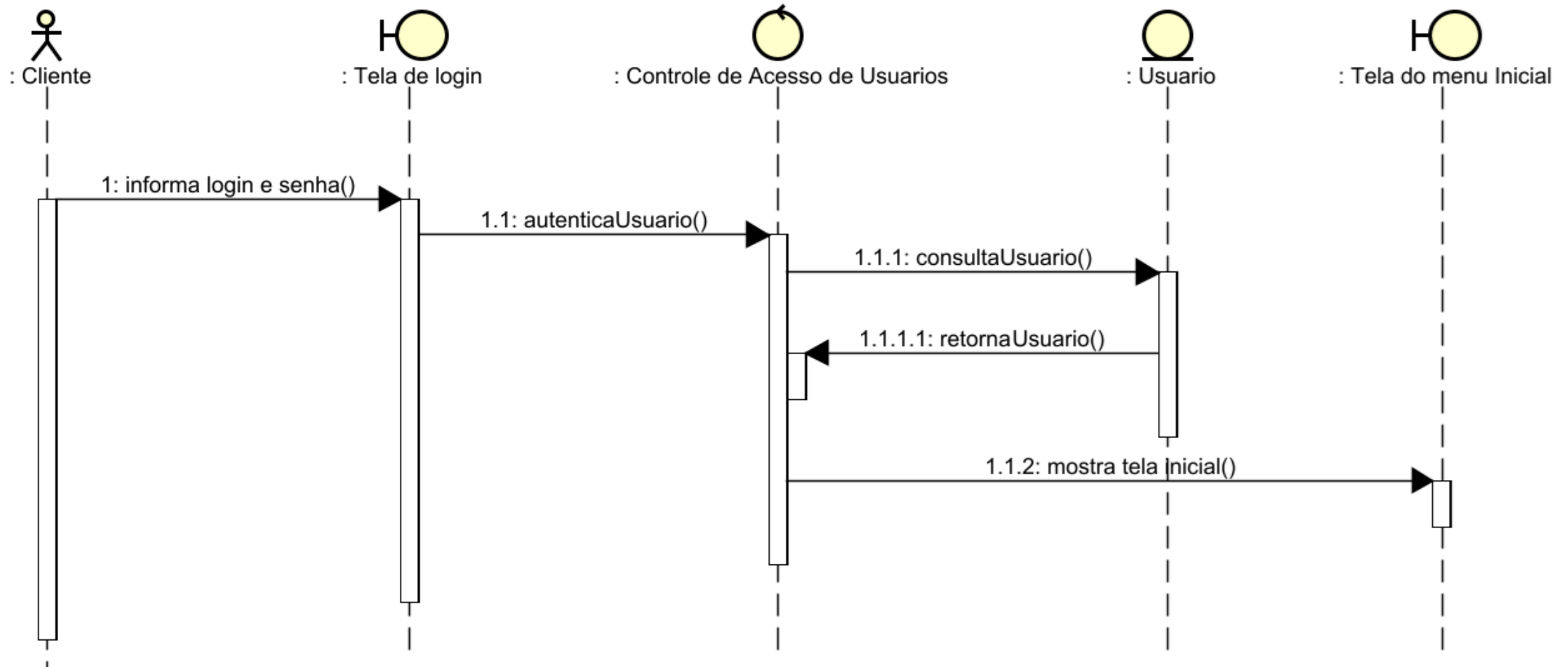
Importância da Categorização BCE

- A categorização BCE parte do princípio de que cada objeto em um sistema é especialista em realizar um de três tipos de tarefa:
 - se comunicar com atores (fronteira)
 - manter as informações (entidade)
 - coordenar a realização de um caso de uso (controle)
- A categorização BCE é uma “receita de bolo” para identificar objetos participantes da realização de um caso de uso

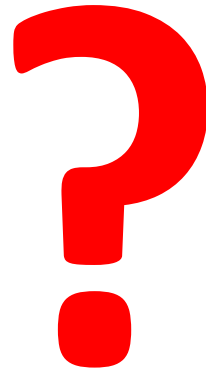
Importância da Categorização BCE

- A importância dessa categorização está relacionada à capacidade de adaptação a eventuais mudanças
 - Se cada objeto tem atribuições específicas dentro do sistema, mudanças podem ser menos complexas e mais localizadas
 - Uma modificação em uma parte do sistema tem menos possibilidades de resultar em mudanças em outras partes

Exemplo com Diagrama de Sequência



Dúvidas

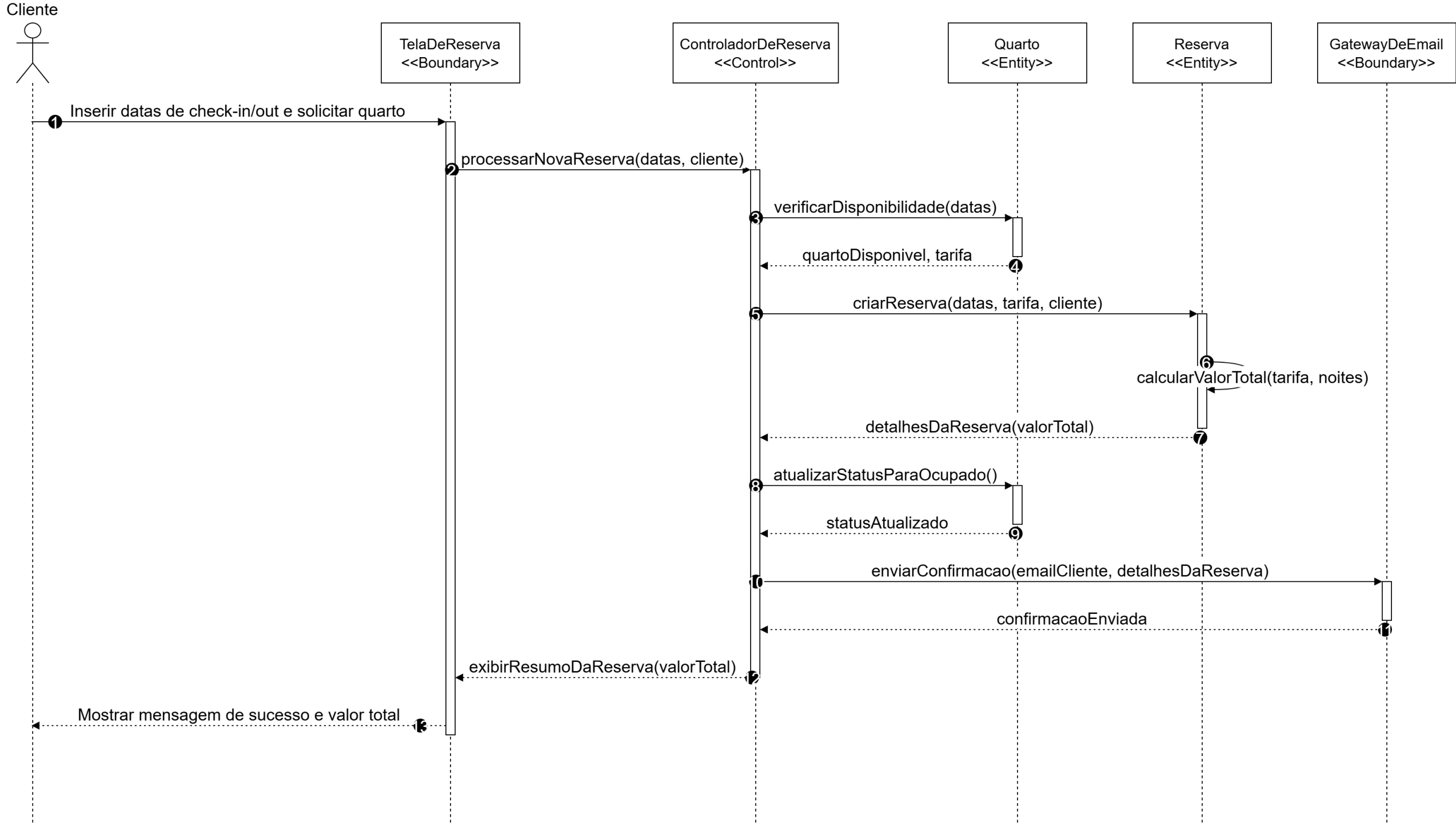




Exercícios

- Aplique a Categorização BCE para o Sistema de Reserva de Hotel

O Sistema de Reserva de Hotel permite que clientes realizem reservas de quartos. Cada reserva possui datas de check-in e check-out. O sistema calcula o valor total baseado na tarifa do quarto e noites de hospedagem. Funcionários do hotel podem gerenciar a disponibilidade dos quartos. O sistema envia confirmações por email para clientes.



Buscar Disponibilidade:

O Cliente seleciona buscar quartos

O Cliente informa as datas desejadas

O Sistema verifica a disponibilidade dos quartos e exibe uma lista de quartos disponíveis no período

Selecionar Quarto e Iniciar Reserva:

O Cliente seleciona um quarto

O Cliente preenche as informações para a reserva

O Sistema cria uma reserva prévia para o quarto selecionado

Calcular Valor Total:

O Sistema calcula o valor total da reserva ($\text{valor} = \text{tarifa_diária} * \text{número_de_noites}$)

Processar Pagamento:

O Sistema apresenta o valor da reserva ao Cliente

O Cliente informa os dados de pagamento

O Sistema processa o pagamento

O Sistema confirma a reserva

Enviar Confirmação:

O Sistema envia os detalhes da reserva para o cliente via e-mail

Entity (Entidade)

- **Quarto** - Representa cada quarto do hotel com suas características
- **Reserva** - Armazena todos os dados da reserva (datas, cliente, quarto, status)
- **Cliente** - Armazena as informações do cliente
- **Tarifa** - Contém as tarifas diárias dos quartos
- **Pagamento** - Armazena os dados e status do pagamento
- **Disponibilidade** - Gerencia o calendário de disponibilidade dos quartos

Boundary (Limite)

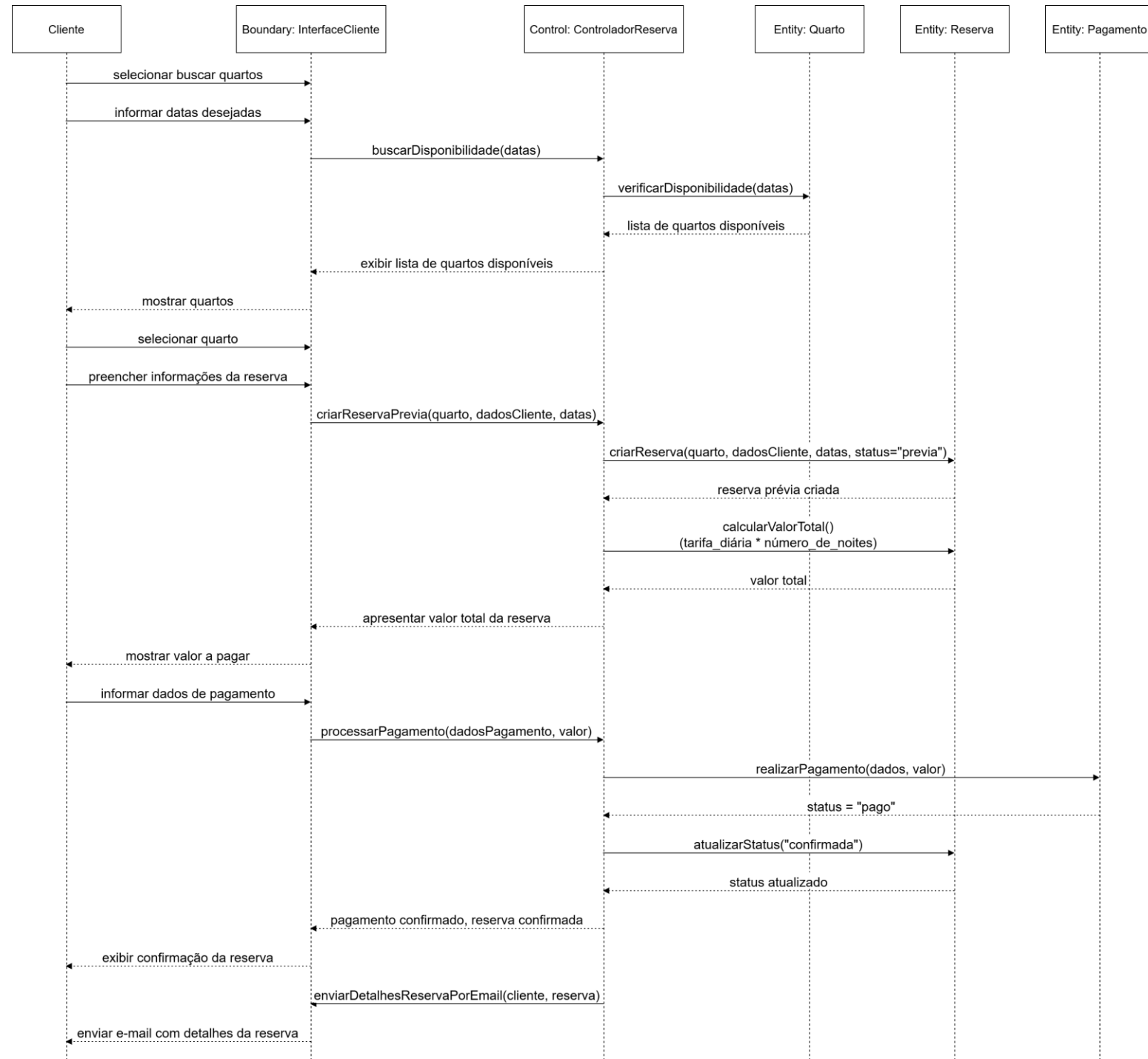
- **Interface de Busca de Quartos** - Onde o cliente informa as datas desejadas
- **Interface de Listagem de Quartos** - Exibe os quartos disponíveis
- **Interface de Reserva** - Onde o cliente preenche informações pessoais
- **Interface de Pagamento** - Onde o cliente insere dados de pagamento
- **Sistema de Email** - Para envio da confirmação da reserva

Control (Controle)

- **Gerenciador de Disponibilidade** - Verifica a disponibilidade dos quartos no período
- **Processador de Reservas** - Cria a reserva prévia e gerencia o fluxo
- **Calculador de Valores** - Calcula o valor total ($\text{tarifa_diária} \times \text{número_de_noites}$)
- **Processador de Pagamentos** - Processa o pagamento e confirma a reserva
- **Coordenador de Confirmação** - Gerencia o envio dos detalhes da reserva

Fluxo BCE:

1. **Boundary** coleta datas → **Control** verifica disponibilidade → **Entity** Quarto e Disponibilidade
2. **Boundary** coleta dados reserva → **Control** cria reserva → **Entity** Reserva e Cliente
3. **Control** calcula valor → **Entity** Tarifa e Reserva
4. **Boundary** coleta pagamento → **Control** processa → **Entity** Pagamento e Reserva
5. **Control** envia confirmação → **Boundary** sistema de email



Modelagem de Classes de Projeto

Introdução

- Modelo de classes de projeto é resultante de refinamentos no modelo de classes de análise
- Construído em paralelo com o modelo de interações
- Construção do modelo de interações gera informações para a transformação do modelo de classes de análise no modelos de classes de projeto
- Modelo de classes de projeto contém detalhes úteis para a implementação das classes nele contidas

Introdução

- Aspectos a serem considerados na fase de projeto para a modelagem de classes:
 - Estudo de novos elementos do diagrama de classes necessários à construção do modelo de projeto
 - Descrever transformações pelas quais passam as classes e suas propriedades com o objetivo de transformar o modelo de classes análise no modelo de classes de projeto
 - Adição de novas classes ao modelo
 - Especificação de atributos, operações e de associações
 - Descrever refinamentos e conceitos relacionados à herança, que surgem durante a modelagem de classes de projeto
 - classes abstratas, interfaces, polimorfismo e padrões de projeto
 - Utilização de padrões de projeto (*design patterns*)



Especificação de Classes de Fronteira

- Não devemos atribuir a essas classes responsabilidades relativas à lógica do negócio
 - Classes de fronteira devem apenas servir como um ponto de captação de informações, ou de apresentação de informações que o sistema processou
 - A única inteligência que essas classes devem ter é a que permite a elas realizarem a comunicação com o ambiente do sistema



Especificação de Classes de Fronteira

- Razões:
 - Se o sistema tiver que ser implantado em outro ambiente, as modificações resultantes sobre seu funcionamento propriamente dito seriam mínimas
 - Sistema pode dar suporte a diversas formas de interação com seu ambiente
 - Ex: uma interface gráfica e uma interface de texto
 - Essa separação resulta em uma melhor coesão



Especificação de Classes de Fronteira

- Durante a análise, considera-se que há uma única classe de fronteira para cada ator
 - No projeto, algumas dessas classes podem resultar em várias outras
- Interface com seres humanos:
 - Projeto da interface gráfica produz o detalhamento das classes
- Outros sistemas ou equipamentos: definir uma ou mais classes para encapsular o protocolo de comunicação
 - É usual a definição de um subsistema para representar a comunicação com outros sistemas de software ou com equipamentos
- O projeto de objetos de fronteira é altamente dependente da natureza do ambiente



Especificação de Classes de Fronteira

- Clientes WEB clássicos
 - Classes de fronteira são representadas por páginas HTML que, muitas vezes, representam sites dinâmicos
- Clientes móveis
 - Classes de fronteira implementam algum protocolo específico com o ambiente
 - Ex: WML (*Wireless Markup Language*)
- Clientes *stand-alone*
 - Nesse caso, é recomendável que os desenvolvedores pesquisem os recursos fornecidos pelo ambiente de programação sendo utilizado
 - Ex: Swing/JFC da linguagem Java
- Serviços WEB (*WEB services*)
 - Um serviço WEB é uma forma de permitir que uma aplicação forneça seus serviços (funcionalidades) através da Internet



Especificação de Classes de Entidade

- A maioria das classes de entidade normalmente permanece na passagem da análise ao projeto
 - Classes de entidade são normalmente as primeiras classes a serem identificadas na análise de domínio
- Durante o projeto, um aspecto importante a considerar sobre classes de entidade é identificar quais delas geram objetos que devem ser persistentes
 - Para essas classes, o seu mapeamento para algum mecanismo de armazenamento persistente deve ser definido
- Um aspecto importante é a forma de representar associações, agregações e composições entre objetos de entidade
 - Essa representação é função da navegabilidade e da multiplicidade definidas para a associação



Especificação de Classes de Entidade

- Outro aspecto relevante para classes de entidade é modo como podemos identificar cada um de seus objetos unicamente
 - Isso porque, principalmente em sistemas de informação, objetos de entidade devem ser armazenados de modo persistente
 - Ex: um objeto da classe Aluno é unicamente identificado pelo valor de sua matrícula (um atributo do domínio)



Especificação de Classes de Entidade

- A manipulação dos diversos atributos identificadores possíveis em uma classes pode ser bastante trabalhosa
- Para evitar isso, um identificador de implementação é criado, que não tem correspondente com atributo algum do domínio
 - Possibilidade de manipular identificadores de maneira uniforme e eficiente
 - Maior facilidade quando objetos devem ser mapeados para um SGBDR



Especificação de Classes de Controle

- No projeto devemos identificar a real utilidade das classes de controle
 - Em casos de uso simples (ex: manutenção de dados), classes de controle não são realmente necessárias
 - Neste caso, classes de fronteira podem repassar os dados fornecidos pelos atores diretamente para as classes de entidade correspondentes
- É comum a situação em que uma classe de controle de análise ser transformada em duas ou mais classes no nível de especificação
- No refinamento de qualquer classe proveniente da análise, é possível a aplicação de padrões de projeto (*design patterns*)



Especificação de Classes de Controle

- Normalmente, cada classe de controle deve ser particionada em duas ou mais outras classes para controlar diversos aspectos da solução
 - Objetivo: evitar a criação de uma única classe com baixa coesão e alto acoplamento
- Alguns exemplos dos aspectos de uma aplicação cuja coordenação é de responsabilidade das classes de controle:
 - Produção de valores para preenchimento de controles da interface gráfica
 - Autenticação de usuários
 - Controle de acesso a funcionalidades do sistema, etc



Especificação de Classes de Controle

- Tipo comum de controlador: controlador de **caso de uso**, responsável pela coordenação da realização de um caso de uso
- Responsabilidades esperadas de um controlador de caso de uso:
 - Coordenar a realização de um caso de uso do sistema
 - Servir como canal de comunicação entre objetos de fronteira e objetos de entidade
 - Se comunicar com outros controladores, quando necessário
 - Mapear ações do usuário (ou atores de uma forma geral) para atualizações ou mensagens a serem enviadas a objetos de entidade
 - Estar apto a manipular exceções provenientes das classes de entidades

Especificação de Outras Classes

- Além do refinamento de classes preexistentes, diversos outros aspectos demanda a identificação de novas classe durante o projeto
 - Persistência de objetos
 - Distribuição e comunicação (e.g., RMI, CORBA, DCOM, WEB)
 - Autenticação/Autorização
 - Logging
 - Configurações
 - Threads
 - Classes para testes (*Test Driven Development*)
 - Uso de bibliotecas, componentes e frameworks
- Conclusão: a tarefa de identificação (reuso?) de classes não termina na análise

Refinamento de Atributos e Métodos

- Atributos e métodos de uma classe a habilitam a cumprir com suas responsabilidades
- Atributos: permitem que uma classe armazene informações necessárias à realização de suas tarefas
- Métodos: funções que manipulam os valores do atributos, com o objetivo de atender às mensagens que o objeto recebe
- Especificação completa para atributos/operações deve seguir as sintaxes definidas pela UML:

[/] [visibilidade] nome [multiplicidade] [: tipo] [= valor-inicial]

[visibilidade] nome [(parâmetros)] [: tipo-retorno] [{propriedades}]

Sintaxe para Atributos e Operações

Carro
- modelo : String - quilometragem : int = 0 - cor : Cor = Cor.Branco - valor : Quantia = 0.0 - tipo : TipoCarro
+ getModelo() : String + setModelo(modelo : String) : void + getQuilometragem() : String + setQuilometragem(quilometragem : int) : void + getCor() : Cor + setCor(cor : Cor) : void + setValor(Quantia : int) : void + getValor() : Quantia

Cliente
+obterNome() : String +definirNome(in umNome : String) +obterDataNascimento() : Data +definirDataNascimento(in umaData : Data) +obterTelefone() : String +definirTelefone(in umTelefone : String) +obterLimiteCrédito() : Moeda +definirLimiteCrédito(in umLimiteCrédito : float) +obterIdade() : int +obterQuantidadeClientes() : <u>int</u> +obterIdadeMédia() : <u>float</u>

Cliente
#nome : String -dataNascimento : Data -telefone : String #/idade : int #limiteCrédito : Moeda = 500.0 <u>-quantidadeClientes : int</u> <u>-idadeMédia : float</u>

- Obs.: classes utilitárias podem ser utilizadas como tipos para atributos. (e.g., Moeda, Quantia, TipoCarro, Endereco)

Implementação de Associações

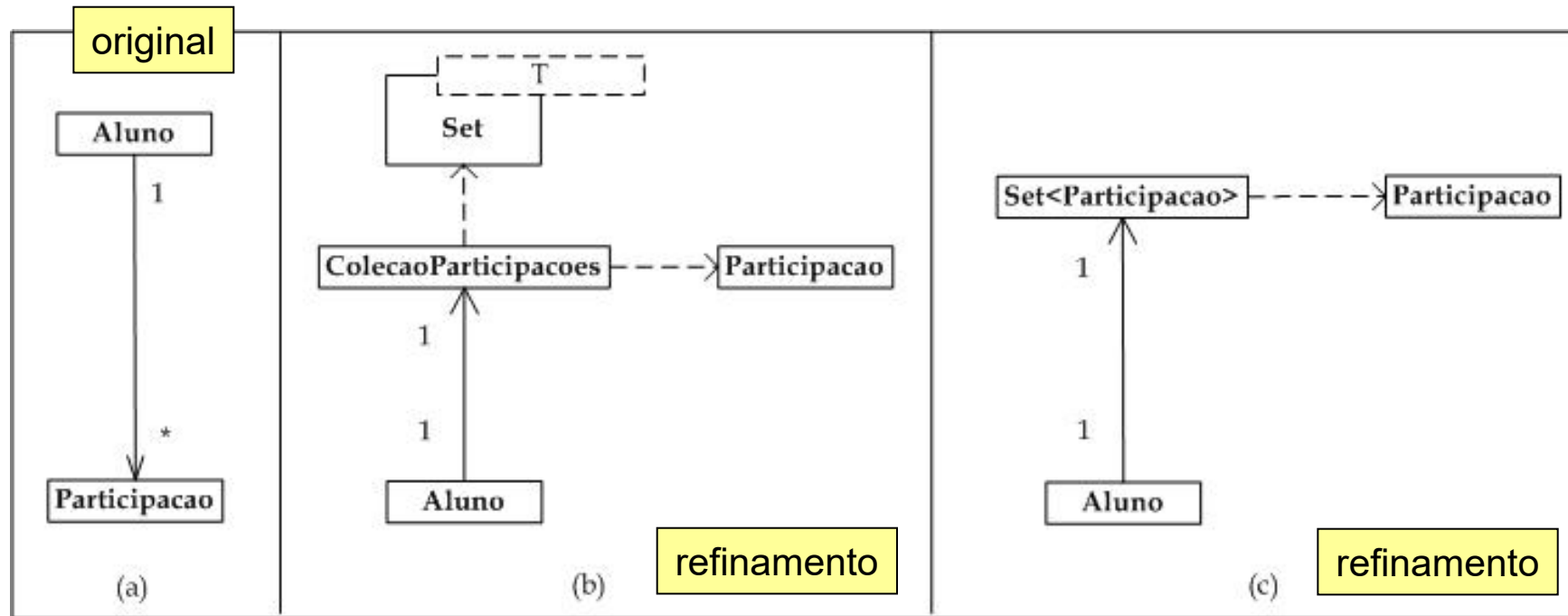
- Três casos: 1:1, 1:N e N:M
- Associação 1:1 entre duas classes A e B:
 - Se a navegabilidade é unidirecional no sentido de A para B, é definido um atributo do tipo B na classe A
 - Se a navegabilidade é bidirecional, podemos aplicar o procedimento acima para as duas classes
- Associação 1:N ou N:M entre duas classes A e B:
 - São utilizados atributos cujos tipos representam coleções de elementos
 - Comum o uso de classes parametrizadas
 - Ideia básica: definir uma classe parametrizada cujo parâmetro é a classe correspondente ao lado muitos da associação
 - Caso N:M é bastante semelhante ao refinamento das associações um para muitos

Conectividade 1:1



```
public class Professor {  
    private GradeDisciplinas grade;  
    ...  
}
```

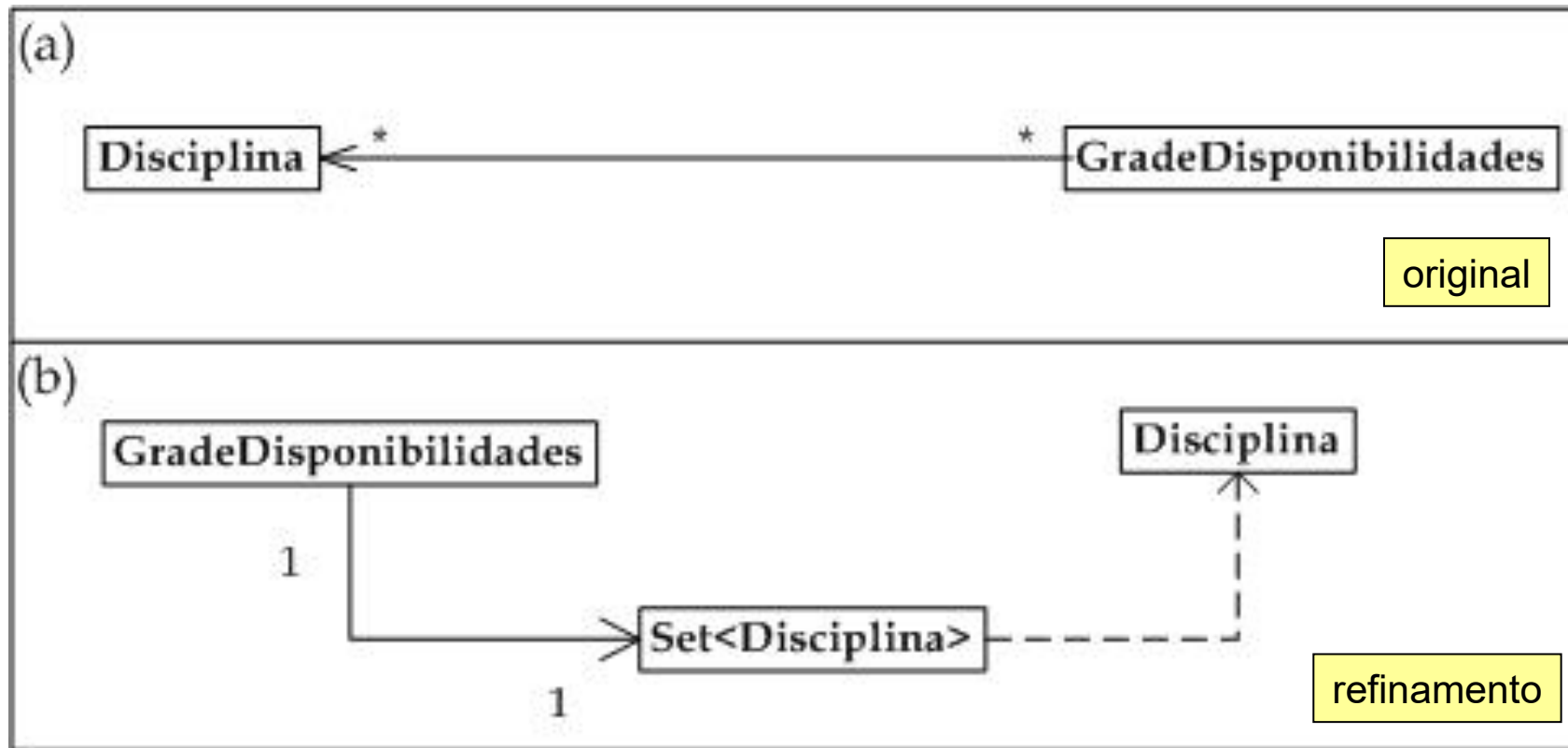
Conectividade 1:N



Conectividade 1:N (cont)

```
public class Aluno {  
    private Set<Participacao> participacoes;  
    ...  
  
    public boolean adicionarParticipacao(Participacao p) {  
        ...  
    }  
  
    public boolean removerParticipacao(Participacao p) {  
        ...  
    }  
}
```

Conectividade N:M



Padrões

Padrões de Análise

- Após produzir diversos modelos para um mesmo domínio, é natural que um modelador comece a identificar características comuns entre esses modelos
- Em particular, um mesmo conjunto de classes ou colaborações entre objetos costuma recorrer, com algumas pequenas diferenças, em todos os sistemas desenvolvidos para o domínio em questão

Padrões de Análise

Quantos modelos de classes já foram construídos que possuem os conceitos Cliente, Produto, Fornecedor, Departamento, etc?

Quantos outros já foram construídos que possuíam os conceitos Ordem de Compra, Ordem de Venda, Orçamento, Contrato, etc?

Padrões de Análise

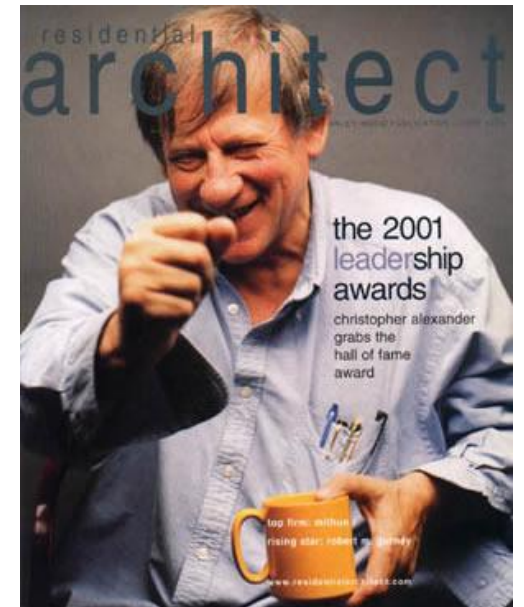
- Identificação de características comuns >>> consequência da experiência do modelador em determinado domínio de problema
- Ao reconhecer processos e estruturas comuns em um domínio, o modelador pode descrever (catalogar) a essência dos mesmos, dando origem a padrões de software
- Quando o problema correspondente ao descrito em um padrão de software acontecer novamente, um modelador pode utilizar a solução descrita no catálogo
- A aplicação desse processo permite que o desenvolvimento de determinado aspecto de um sistema seja feito de forma mais rápida e menos suscetível a erros

Padrões de Análise

- Padrão de software pode ser definido como uma descrição essencial de um problema recorrente no desenvolvimento de software

“Cada padrão descreve um problema que ocorre frequentemente no nosso ambiente, e então descreve o núcleo de uma solução para tal problema. Esse núcleo pode ser utilizado um milhão de vezes, sem que haja duas formas de utilização iguais.”

Christopher Alexander



Padrões de Análise

- Padrões de software podem ser utilizados em diversas atividades e existem em diversos níveis de abstração
 - Padrões de Análise (Analysis Patterns)
 - Padrões de Projeto (Design Patterns)
 - Padrões Arquiteturais (Architectural Patterns)
 - Idiomas (Idioms)
 - Anti-padrões (Anti-patterns)

Padrões de Análise

- Padrões de software vêm sendo estudados por anos e são atualmente utilizados em diversas fases do desenvolvimento
- Padrões de software utilizados na fase de análise de um sistema são chamados padrões de análise
- Um padrão de análise normalmente é composto de um fragmento de diagrama de classes que pode ser customizado para uma situação de modelagem em particular

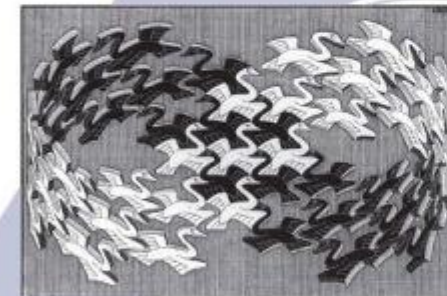
Padrões de Projeto

- Esboço de uma solução reusável para um problema comumente encontrado em um contexto particular
- Texto clássico
- Gang of Four
- 23 padrões catalogados

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Condon Art - Baam - Holland. All rights reserved.

Foreword by Grady Booch

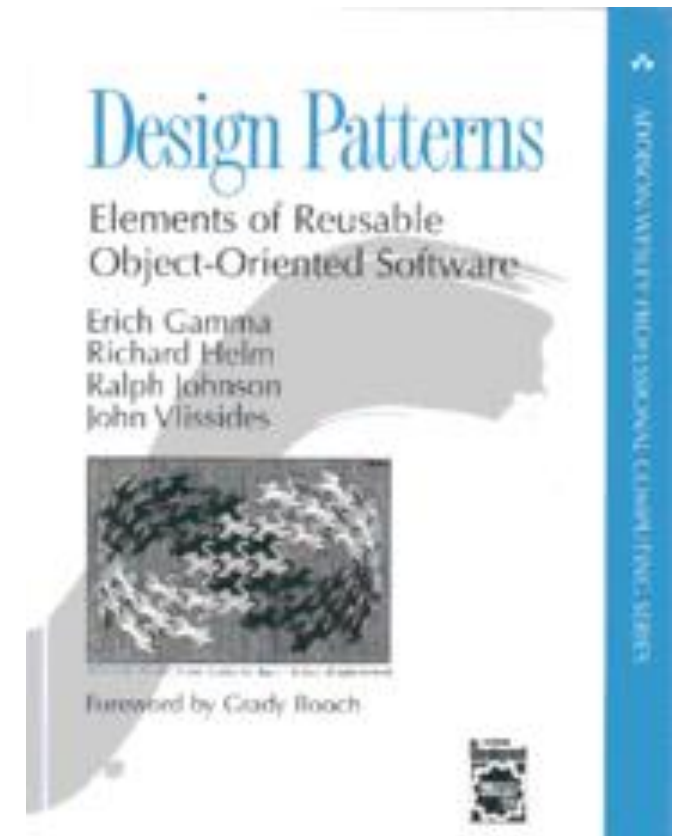


Padrões GoF

- Divididos em três categorias:
 - Criacionais:
 - Procuram separar a operação de uma aplicação de como os seus objetos são criados
 - Estruturais:
 - Provêm generalidade para que a estrutura da solução possa ser estendida no futuro
 - Comportamentais:
 - Utilizam herança para distribuir o comportamento entre subclasses, ou agregação e composição para construir comportamento complexo a partir de componentes mais simples

Padrões GoF

Criacionais	Estruturais	Comportamentais
Abstract Factory Builder Factory Method Prototype Singleton	Adapter Bridge Composite Decorator Façade Flyweight Proxy	Chain of Responsibility Command Interpreter Iterator Mediator Memento Observer State Strategy Template Method Visitor



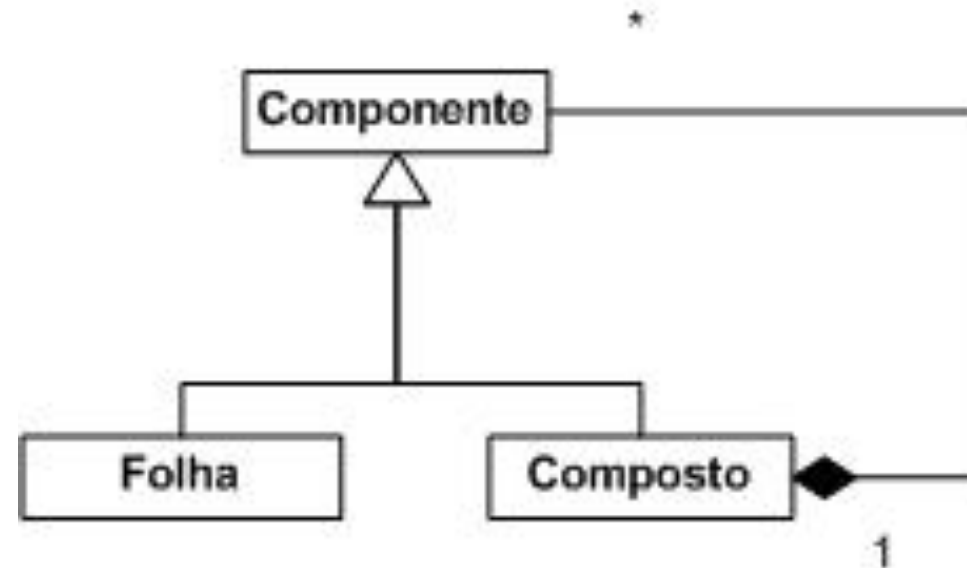
Padrão Composite

- Problema: como criar objetos utilizando partes de tal forma que tanto o objeto todo quanto os objetos parte forneçam a mesma interface para os seus clientes?
- Contexto: em BD, o padrão Composite pode ser utilizado para solucionar o problema de representar uma hierarquia de composição recursiva entre entidades
- Ex: montagem de peças
 - Uma peça é composta de diversas outras peças, e essas peças componentes são elas próprias compostas, e assim por diante

Padrão Composite

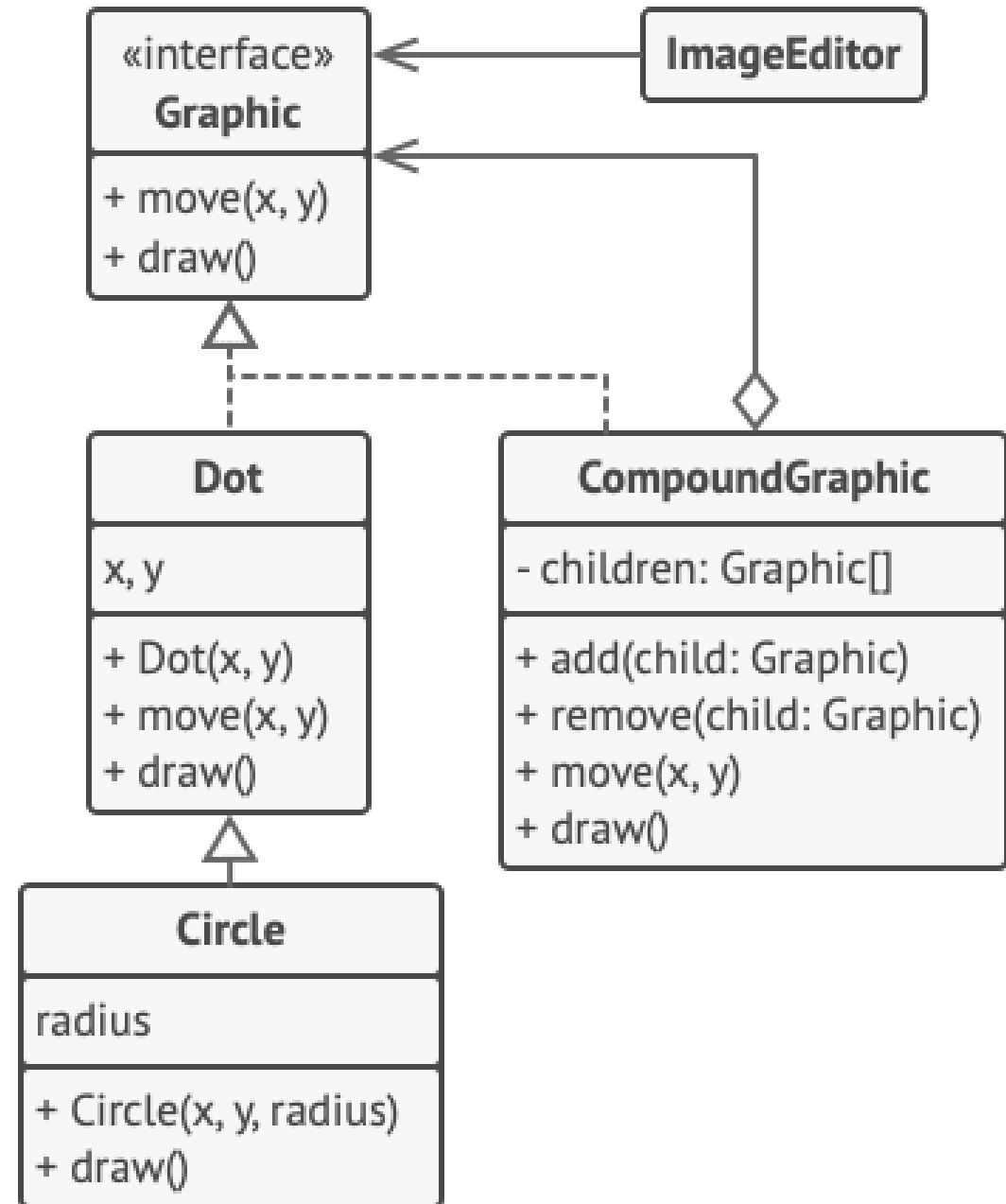
- Componente é uma classe abstrata
- Dois tipos de componentes: Folha e Composto
- Uma Folha não possui componentes, enquanto que um Composto possui
- Associação entre Composto e Componente representa a hierarquia de composição recursiva:
 - Um composto possui diversos componentes, que podem ser ou folhas ou outros compostos

Exemplo 1 - Padrão Composite



Exemplo 1

Padrão Composite



Padrão Metamodel

- Considere um conjunto de itens
- Cada item desse conjunto possui várias propriedades
- As propriedades de cada item podem ser diferentes das propriedades dos outros itens do conjunto
- Casos particulares:
 - Uma classe possui um grupo (fixo) de atributos
 - Uma tabela é formada por uma quantidade (também fixa) de colunas

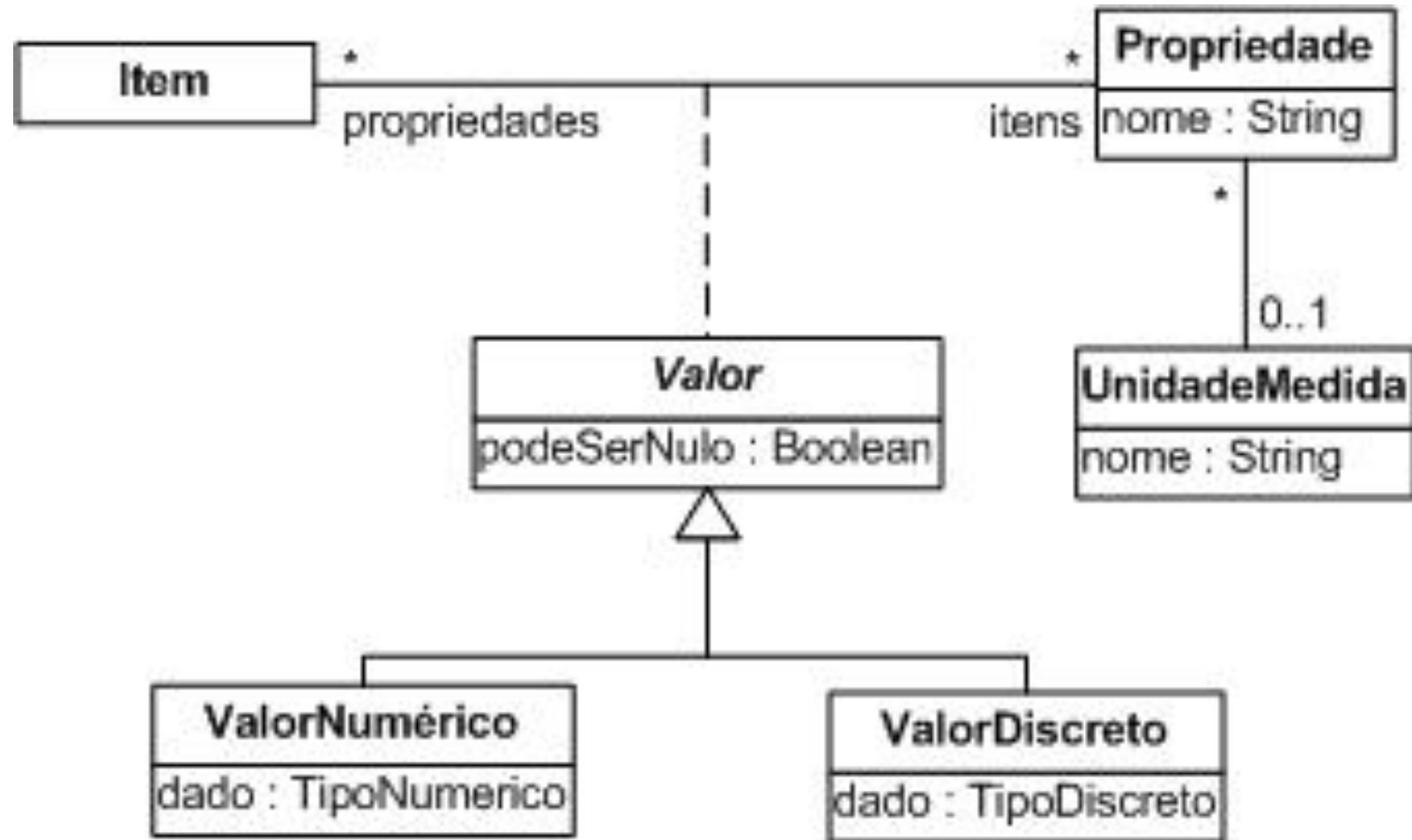
Padrão Metamodel

- Exemplo
 - Considere os seguintes itens: Monitor, Modem, Teclado e Processador
 - Obviamente, as propriedades desses produtos não são as mesmas
 - É desejável criar no modelo de dados um único conceito para representar Produto, sem deixar de modelar as propriedades particulares de cada tipo de produto
 - Uma possível solução para esse problema é criar uma classe denominada Produto e criar diversas subclasses, uma para cada tipo de produto existente
 - Dessa forma, cada subclasse representaria as propriedades particulares do tipo de produto correspondente

Padrão Metamodel

- Desvantagem
 - Novos produtos estão sempre aparecendo e outros desaparecendo, o que levaria a uma modificação na estrutura de modelagem
- Complicação
 - Situações em que as propriedades de um mesmo item podem variar com o tempo
 - Novas propriedades podem ser adicionadas, ou propriedades existentes podem ser removidas
- Consequência
 - Muitas mudanças no esquema do banco de dados, o que poderia acarretar em mudanças nas aplicações que utilizassem esse esquema
- Padrão Metamodel permite a “modificação” da estrutura de um banco de dados sem que o esquema desse banco de dados seja realmente modificado

Exemplo 3 – Padrão Metamodel



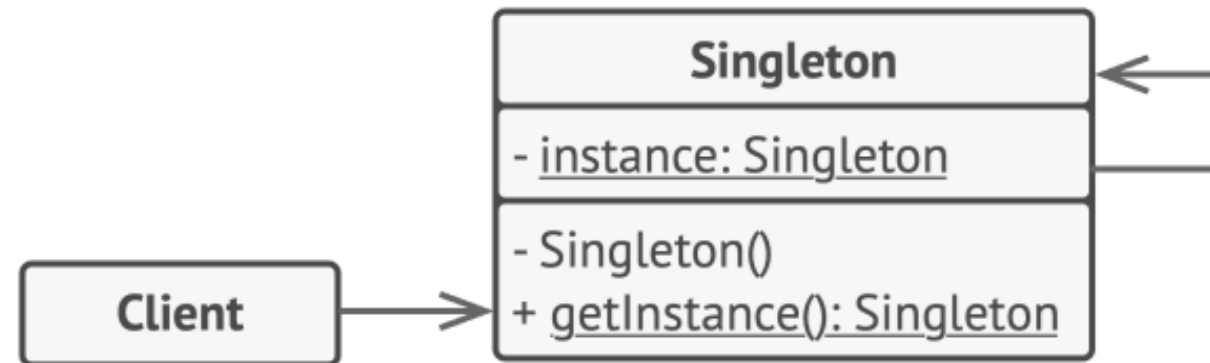
Padrão Singleton

- Padrão de projeto criacional que permite garantir que uma classe tenha apenas uma instância, enquanto provê um ponto de acesso global para essa instância
- Problema:
 - Garantir que uma classe tenha apenas uma única instância
 - Fornece um ponto de acesso global para aquela instância

Padrão Singleton

- Solução:
 - Fazer o construtor padrão privado, para prevenir que outros objetos usem o operador new com a classe singleton
 - Criar um método estático de criação que age como um construtor
 - Esse método chama o construtor privado “por debaixo dos panos” para criar um objeto e o salva em um campo estático
 - Todas as chamadas seguintes para esse método retornam o objeto em cache

Exemplo – Padrão Singleton



1 A classe **Singleton** declara o método estático `getInstance` que retorna a mesma instância de sua própria classe.

O construtor da singleton deve ser escondido do código cliente. Chamando o método `getInstance` deve ser o único modo de obter o objeto singleton.

```
if (instance == null) {
    // Atenção: se você está criando uma
    // aplicação com apoio multithreading,
    // você deve colocar um thread lock aqui.
    instance = new Singleton()
}
return instance
```

```
public final class Singleton {  
    private static Singleton instance;  
    public String value;  
  
    private Singleton(String value) {  
        // The following code emulates slow initialization.  
        try {  
            Thread.sleep(1000);  
        } catch (InterruptedException ex) {  
            ex.printStackTrace();  
        }  
        this.value = value;  
    }  
  
    public static Singleton getInstance(String value) {  
        if (instance == null) {  
            instance = new Singleton(value);  
        }  
        return instance;  
    }  
}
```

```
public class DemoSingleThread {  
    public static void main(String[] args) {  
        System.out.println("If you see the same value, then singleton was reused (yay!)" + "\n" +  
            "If you see different values, then 2 singletons were created (booo!!)" + "\n\n" +  
            "RESULT:" + "\n");  
        Singleton singleton = Singleton.getInstance("FOO");  
        Singleton anotherSingleton = Singleton.getInstance("BAR");  
        System.out.println(singleton.value);  
        System.out.println(anotherSingleton.value);  
    }  
}
```

If you see the same value, then singleton was reused (yay!)

If you see different values, then 2 singletons were created (booo!!)

RESULT:

FOO

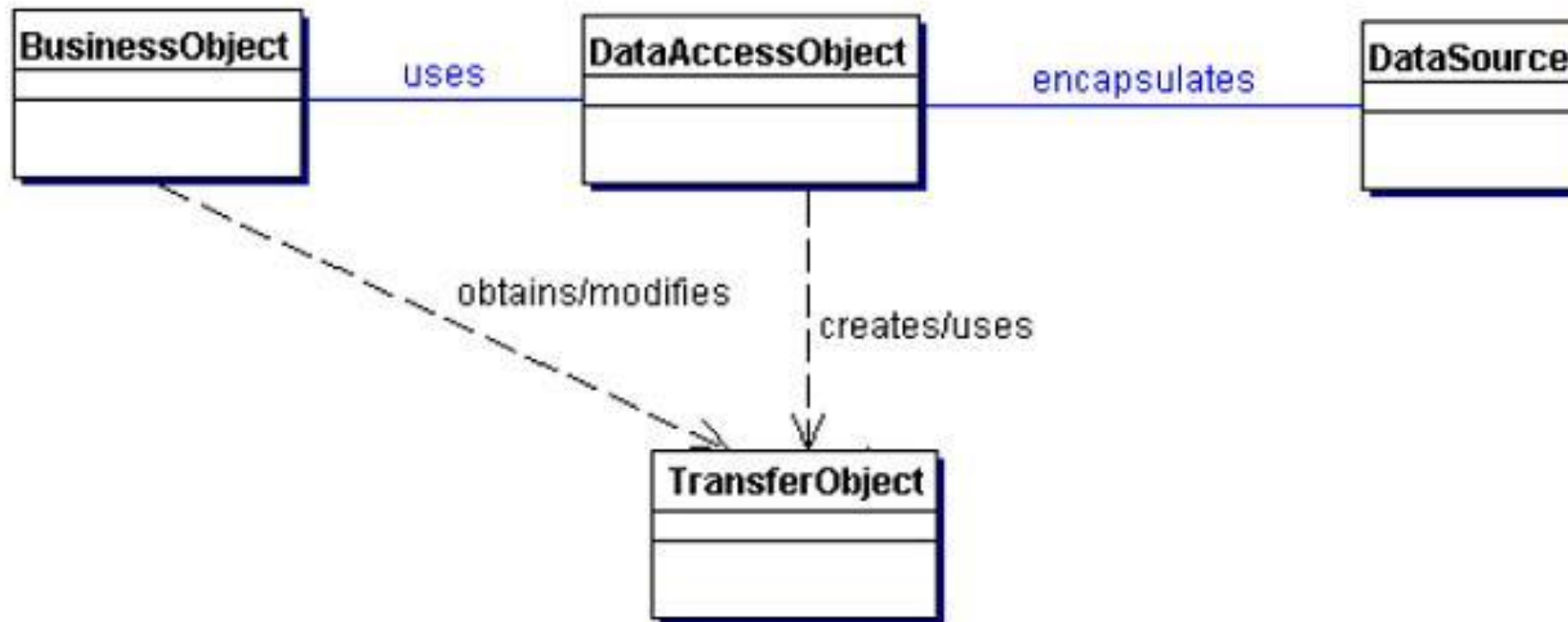
FOO

Padrão DAO

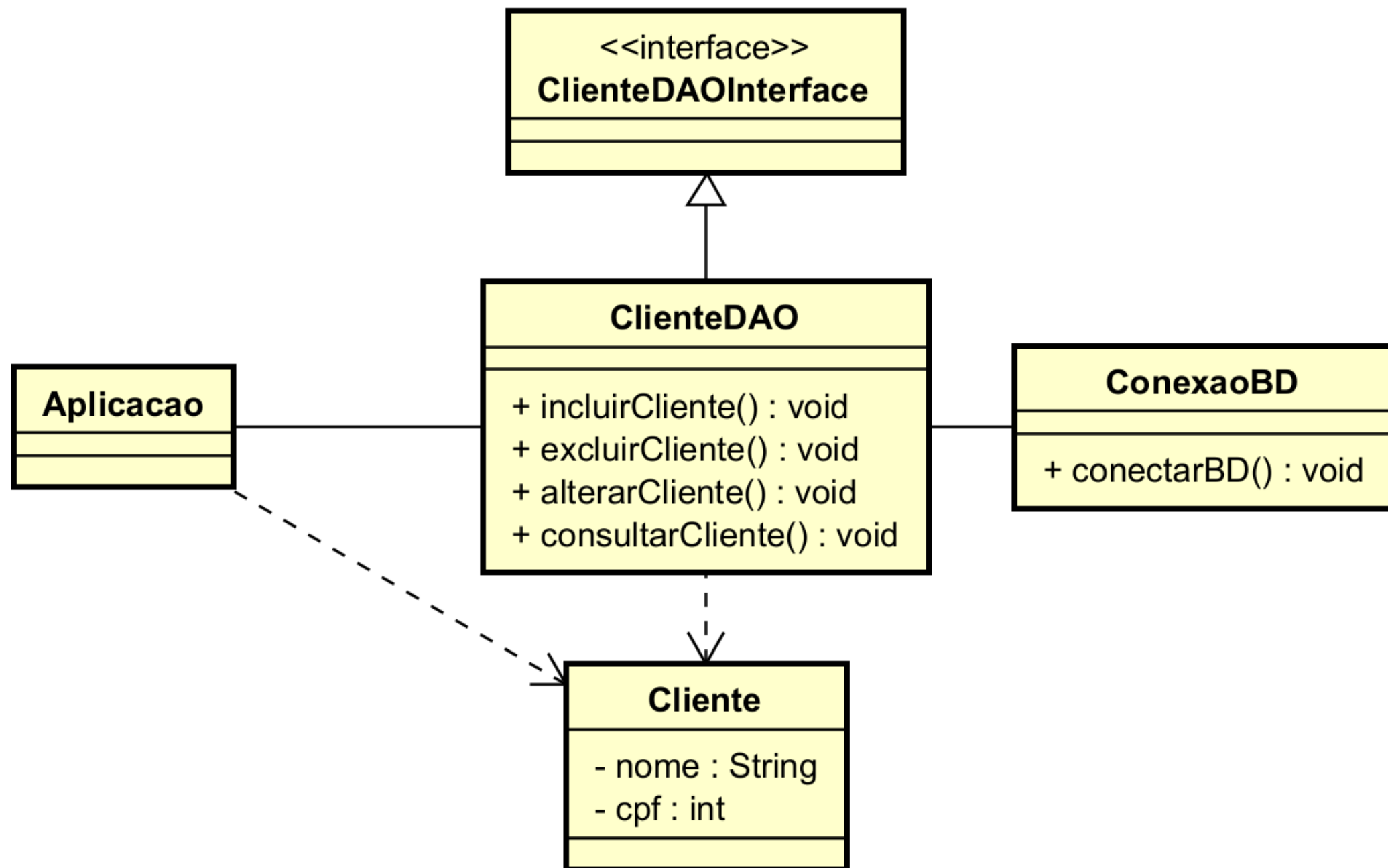
- *Data Access Object*
- Problema:
 - Você deseja encapsular o acesso e a manipulação de dados em uma camada separada
- Permite criar classes de dados independentemente da fonte de dados ser um BD relacional, um arquivo texto, um arquivo XML, etc
- Encapsula mecanismos de acesso a dados e cria uma interface de cliente genérica para fazer o acesso aos dados permitindo que os mecanismos de acesso a dados sejam alterados independentemente do código que utiliza os dados

Padrão DAO

- Todo o acesso aos dados deve ser feita através das classes DAO de forma a se ter o encapsulamento
- Cada instância da DAO é responsável por um objeto de domínio
- O DAO deve ser responsável pelas operações CRUD no domínio
- O DAO não deve ser responsável por transações, sessões ou conexões que devem ser tratados fora do DAO



- A classe DataAccessObject encapsula o acesso aos dados; contém os mapeamentos
- A classe DataSource é a origem dos dados: BD, XML, etc.
- A classe BusinessObject contém a lógica de negócio e usa o objeto DataAccessObject
- Representa o objeto de domínio: Cliente, Produto, etc
- Contém os dados que transitam de/para a fonte de dados



Padrão Fachada / Facade

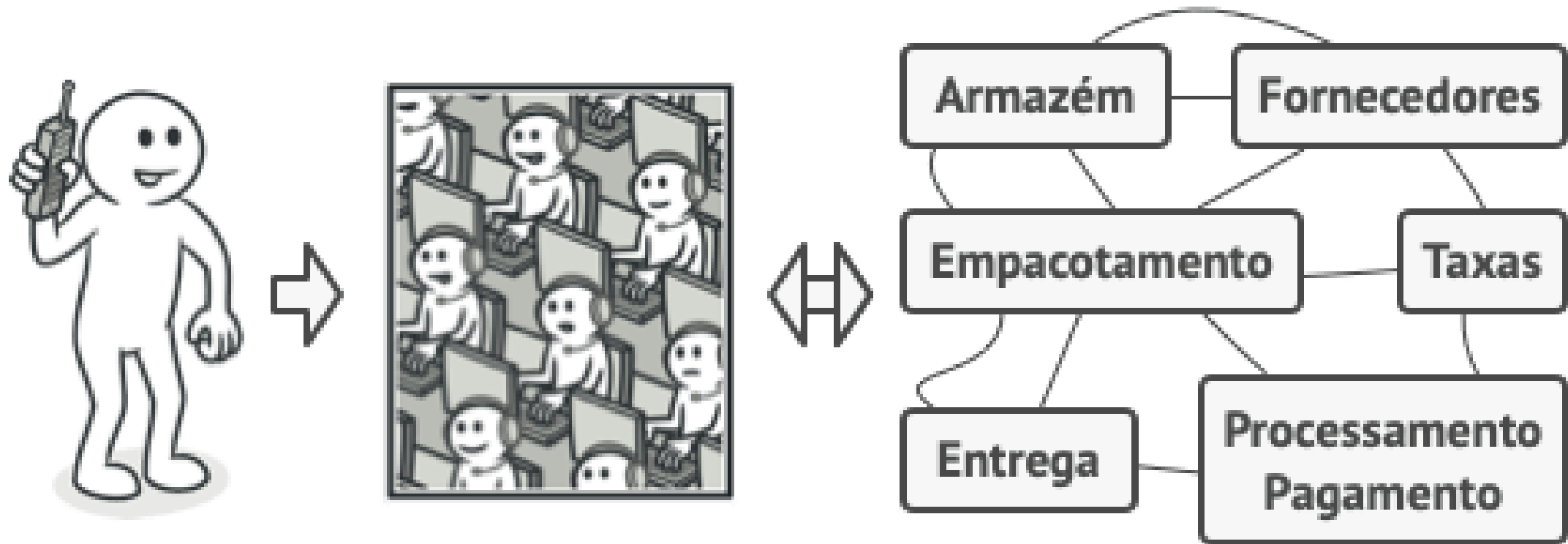
- Propósito
 - Padrão de projeto estrutural que fornece uma interface simplificada para uma biblioteca, um framework, ou qualquer conjunto complexo de classes
- Problema
 - Código precisa funcionar com um amplo conjunto de objetos que pertencem a uma biblioteca ou framework
 - Normalmente você precisaria inicializar todos aqueles objetos, rastrear as dependências, executar métodos na ordem correta, e assim por diante
 - Como resultado, a lógica de negócio de suas classes vai ficar firmemente acoplada aos detalhes de implementação das classes de terceiros, tornando difícil compreendê-lo e mantê-lo

Padrão Fachada / Facade

- Solução
 - Uma “fachada” é uma classe que fornece uma interface simples para um subsistema complexo que contém muitas partes que se movem
 - Uma fachada pode fornecer funcionalidades limitadas em comparação com trabalhar com os subsistemas diretamente
 - Porém ela inclui apenas aquelas funcionalidades que o cliente se importa

Padrão Fachada / Facade

- Solução
 - Útil quando se precisa integrar sua aplicação com uma biblioteca sofisticada que tem dúzias de funcionalidades, mas você precisa de apenas um pouco delas
- Ex:
 - Uma aplicação que carrega vídeos curtos de cachorros para redes sociais poderia potencialmente usar uma biblioteca de conversão de vídeo profissional
 - Porém, tudo que ela realmente precisa é uma classe com um único método `codificar(nomeDoArquivo, formato)`
 - Após criar tal classe e conectá-la com a biblioteca de conversão de vídeo, você terá sua primeira fachada

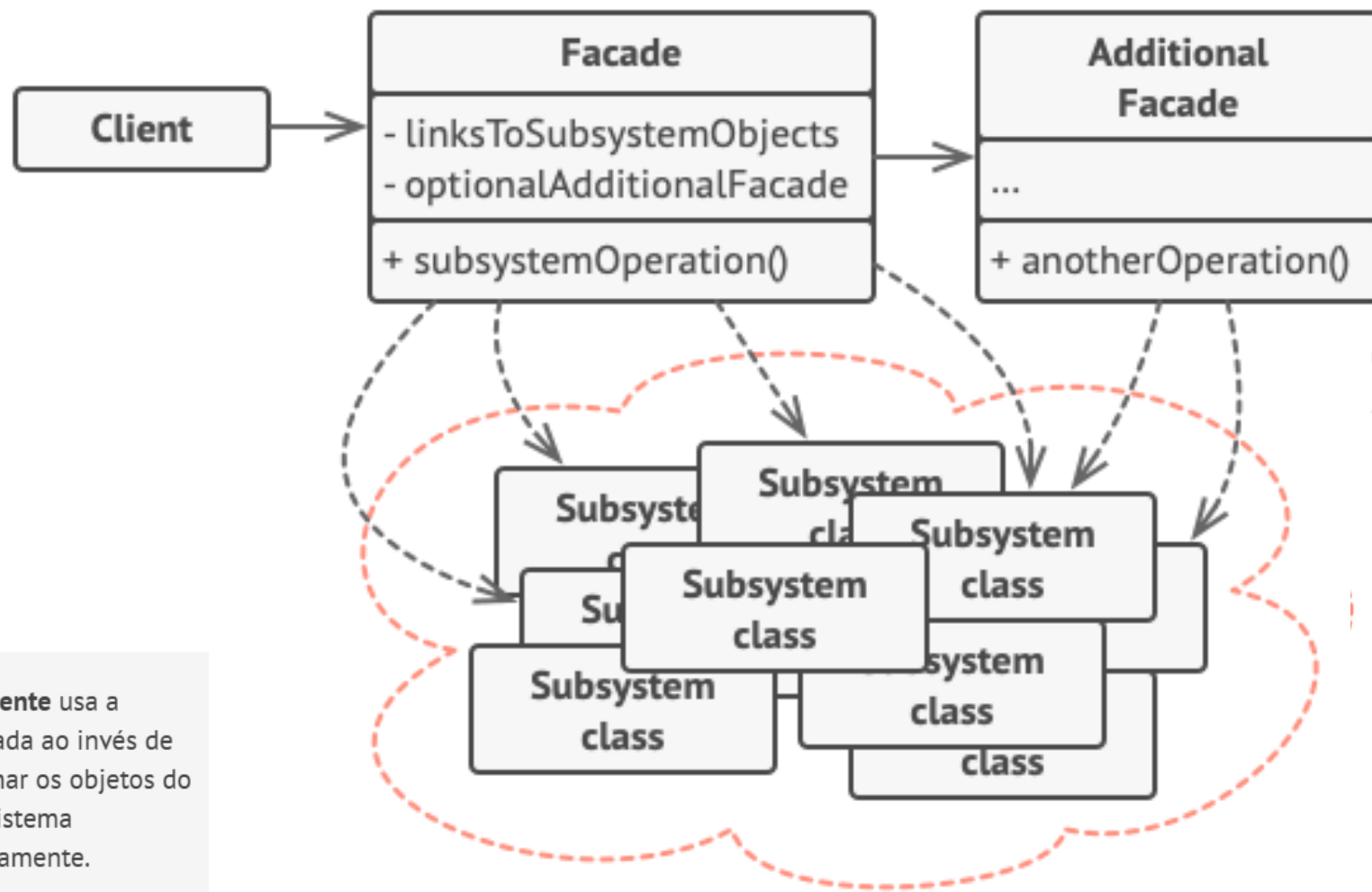


Quando você liga para uma loja para fazer um pedido, um operador é sua fachada para todos os serviços e departamentos da loja

O operador fornece a você uma simples interface de voz para o sistema de pedido, pagamentos, e vários sistemas de entrega

1 A **Fachada** fornece um acesso conveniente para uma parte particular da funcionalidade do subsistema. Ela sabe onde direcionar o pedido do cliente e como operar todas as partes móveis.

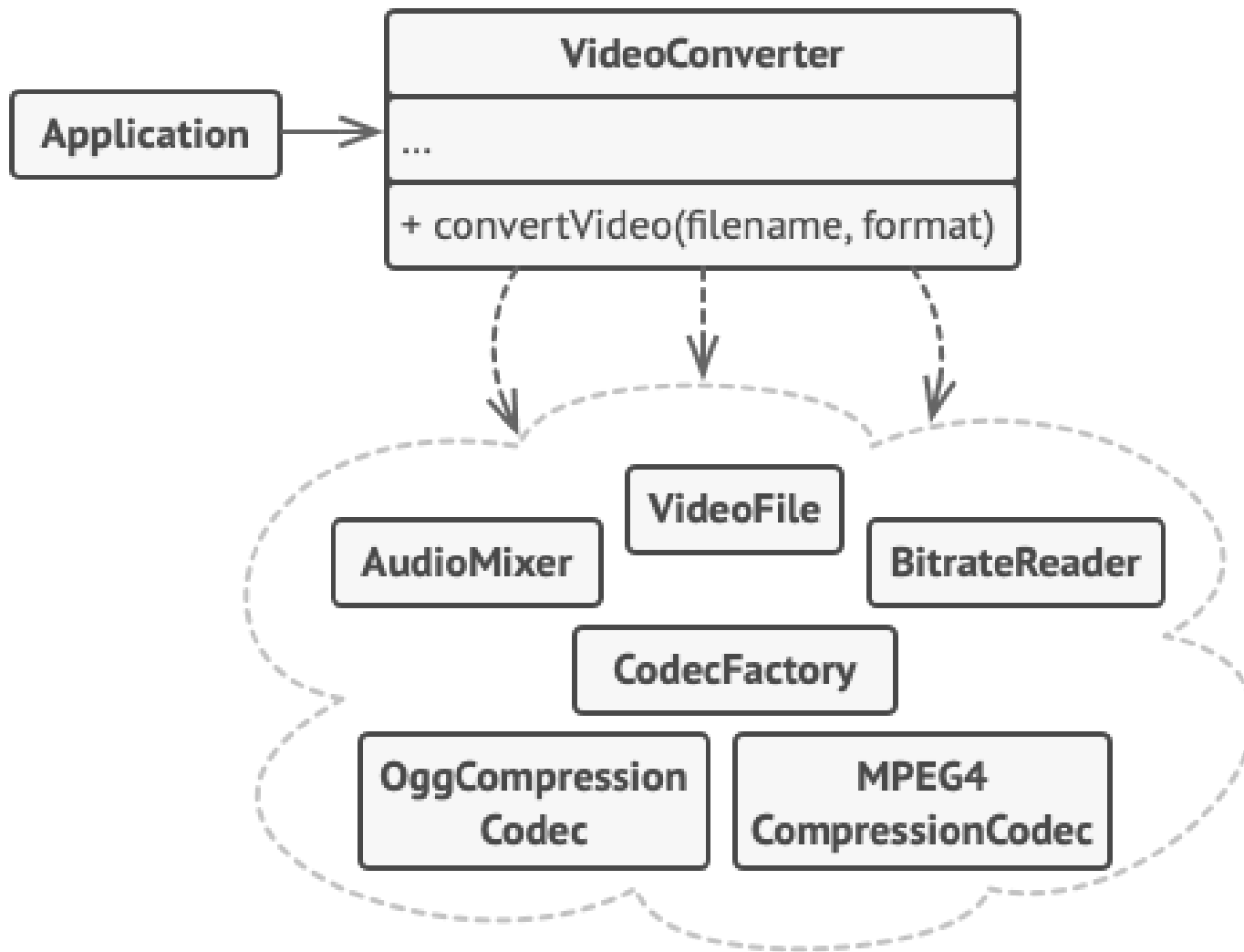
2 Uma classe **Fachada Adicional** pode ser criada para prevenir a poluição de uma única fachada com funcionalidades não relevantes que podem torná-lo mais uma estrutura complexa. Fachadas adicionais podem ser usadas tanto por clientes como por outras fachadas.



4 O **Cliente** usa a fachada ao invés de chamar os objetos do subsistema diretamente.

3 O **Subsistema Complexo** consiste em dúzias de objetos variados. Para tornar todos eles em algo que signifique alguma coisa, você tem que mergulhar fundo nos detalhes de implementação do subsistema, tais como objetos de inicialização na ordem correta e supri-los com dados no formato correto.

As classes do subsistema não estão cientes da existência da fachada. Elas operam dentro do sistema e trabalham entre si diretamente.



Ao invés de fazer o código funcionar com dúzias de classes framework diretamente, você cria a classe fachada que encapsula a funcionalidade e a esconde do resto do código

Essa estrutura também ajuda a minimizar o esforço usando para atualizar futuras versões do framework ou substituí-lo por outro

A única coisa que precisaria mudar em sua aplicação seria a implementação dos métodos da fachada

Neste exemplo, o padrão Facade simplifica a interação com um framework complexo de conversão de vídeo

Exemplo da estrutura do padrão Facade

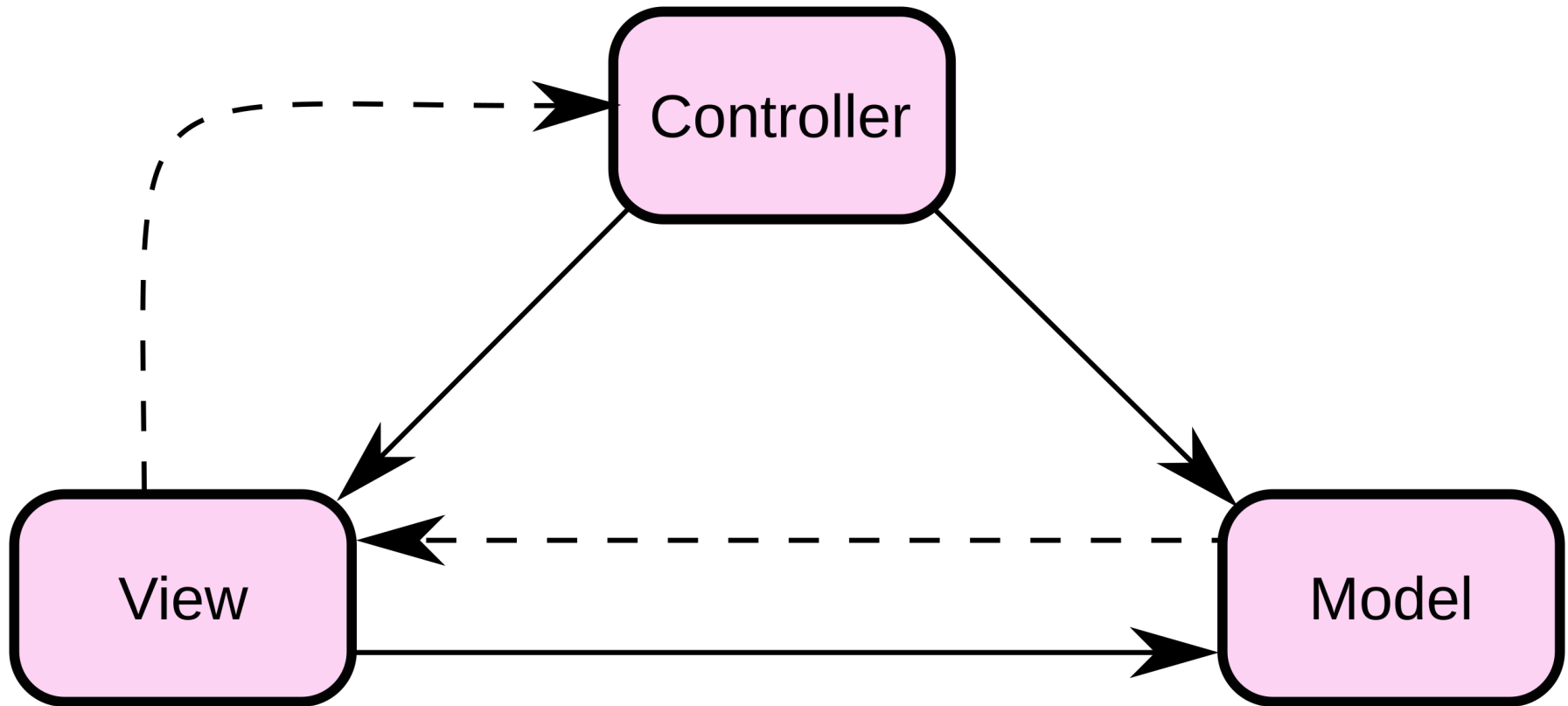
Um exemplo de isolamento de múltiplas dependências dentro de uma única classe fachada

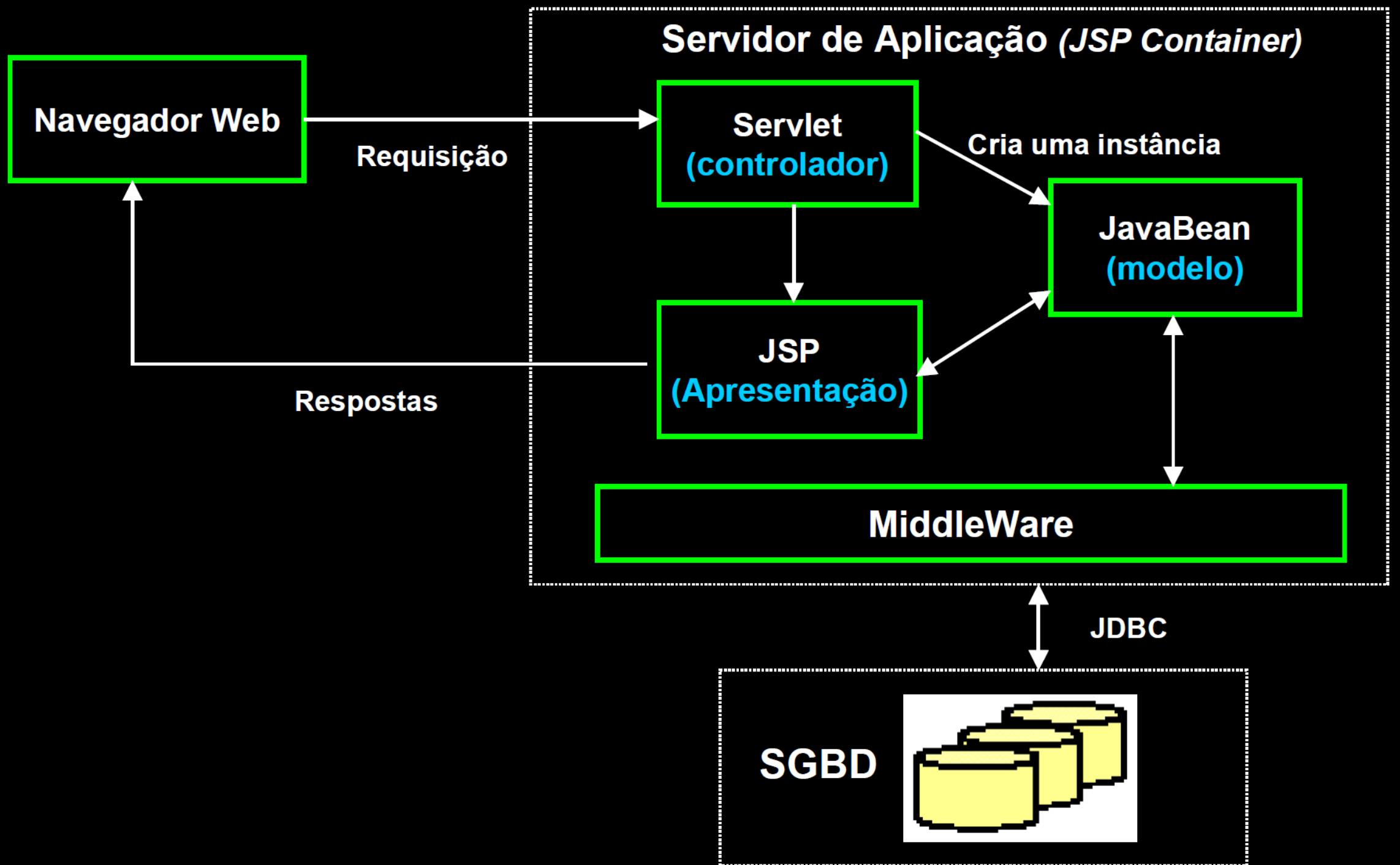
Padrão MVC

- *Model-View-Controller* - Modelo-Visão-Controle
- Focado no reuso de código e separação de conceitos em **três camadas interconectadas**, onde a apresentação dos dados e interação dos usuários (*front-end*) são separados dos métodos que interagem com o banco de dados (*back-end*)

Padrão MVC

- Controlador:
 - Envia comandos para o modelo para atualizar o seu estado (ex: editando um documento)
 - Pode enviar comandos para a visão associada para alterar a apresentação da visão do modelo (ex: percorrendo um documento)
- Modelo:
 - Armazena dados e notifica suas visões e controladores associados quando há uma mudança em seu estado
 - Estas notificações permitem que as visões produzam saídas atualizadas e que os controladores alterem o conjunto de comandos disponíveis
- Visão:
 - Gera uma representação (Visão) dos dados presentes no modelo solicitado, fazendo a exibição dos dados, sendo ela por meio de um html ou xml

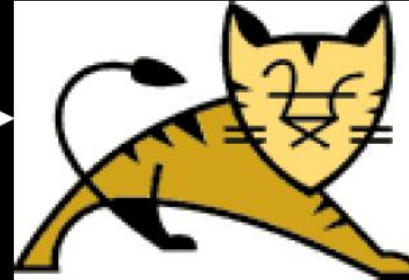






Request

WebServer



Controlador

Forward

JavaServer Page

Response

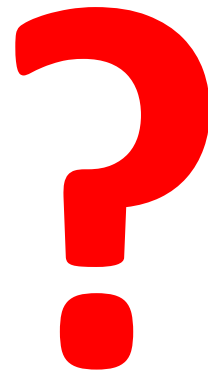
Modelo

Request



Visão

Dúvidas



Referências Bibliográficas

- Freepik
- DeepSeek
- Artigo SQL Magazine 9 - Utilização de Padrões na Modelagem de Dados Artigo da Revista SQL Magazine - Edição 11 - <https://www.devmedia.com.br/artigo-sql-magazine-9-utilizacao-de-padroes-na-modelagem-de-dados/7570>
- Técnicas para Identificação de Classes | Modelagem e Projeto de Software | Aula #10 - <https://www.youtube.com/watch?v=83c0oZQImu0>
- BEZERRA, Eduardo. Princípios de Análise e Projeto de Sistemas com UML. Rio de Janeiro: Elsevier. ISBN-13: 978-85-352-2626-3. 2015.
- Princípios de Análise e Projeto de Sistemas com UML (3a edição) – <https://eic.cefet-rj.br/papsuml3ed/>
- Slides e materiais disponibilizados no site do livro
- The Catalog of Design Patterns - <https://refactoring.guru/design-patterns/catalog>
- Refactoring.Guru - <https://refactoring.guru/pt-br>
- Macoratti.NET - Apresentando o padrão DAO - Data Access Object - https://www.macoratti.net/11/10/pp_dao1.htm
- MVC - <https://pt.wikipedia.org/wiki/MVC>
- MVC - Abordagem Teórico-Prática - Prof. Giuliano Prado M. Giglio, M.Sc - Desenvolvimento de Aplicações Distribuídas - WEB